

# WINA BWANGU FINTECH SYSTEM

## Web Application - Client-Side + Service-Side + Database - Complete Implementation

**Student:** Blessed Chisanga | **ID:** 202202741 | **Live Link:** <https://thewalkingdollar.github.io/wina-bwangu/> | **Vercel API:** <https://wina-bwangu.vercel.app>

**GitHub:** <https://github.com/thewalkingdollar/wina-bwangu> | **Test Plan:** <https://thewalkingdollar.github.io/wina-bwangu/test-plan.html>

## Requirements Compliance Checklist (a-k) - How My System Implements Each Requirement

| Req | Requirement                                                               | How I Implemented                                                                                       | File                        | Status |
|-----|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------|--------|
| a)  | Auto generate TransactionID with text + numbers as Table 3                | I generate IDs as WB0000001, WB0000171 format. Counter increments in backend. Text WB + 7-digit number. | database.js + index.html    | DONE   |
| b)  | Select Mobile Booth from dropdown and display Location                    | I have 6 booths dropdown: Wina1-Lusaka CPD etc. On select, location auto-displays.                      | index.html boothSelect      | DONE   |
| c)  | Dropdown shows Services only at that booth and display Revenue Per Kwacha | I filter services per booth: Wina4 shows 3 services, Wina1 shows 5. Rate auto-displays.                 | index.html filter           | DONE   |
| d)  | Calculate Amounts after Tax and data bars for Tax obligations             | Revenue = Amount * Rate. Net = Amount - Revenue. Data bars show Revenue performance via Chart.js.       | index.html + Chart.js       | DONE   |
| e)  | Populate 170 transactions from Table 3                                    | I populated 170 transactions. Counter starts at 171 for new entries.                                    | database.js                 | DONE   |
| f)  | Dashboard: Cumulative Totals and Remaining from Monthly Limit             | Cumulative per service: Total Used, Monthly Limit, Remaining = Limit - Used.                            | index.html + /api/dashboard | DONE   |
| g)  | Calculate cumulative Revenue for each Mobile booth                        | Per booth revenue calculated and displayed in Booth Performance tab.                                    | index.html boothPerf        | DONE   |
| h)  | Dashboard shows frequency each Service at each booth                      | Frequency counts: e.g., Airtel at Wina1: 45 times. Bar chart shows distribution.                        | index.html frequency        | DONE   |
| i)  | Calculate Total Revenue + Total Capital                                   | Total Revenue = Sum revenues, Total Capital = Sum amounts - shown in cards                              | index.html + /api/dashboard | DONE   |
| j)  | Create Pie Chart for totals in i)                                         | Pie Chart using Chart.js 4.4.0 showing Revenue vs Capital.                                              | index.html doughnut         | DONE   |
| k)  | Brief report 1000 words (20 marks)                                        | Report included in section 8. Covers tools, implications, controls.                                     | This PDF Sec 8              | DONE   |

## Test Plan and Execution - Unit + Regression (23 Tests - 100% PASS)

### Unit Testing - 14 Cases

| ID    | Module             | Input        | Expected    | Status |
|-------|--------------------|--------------|-------------|--------|
| UT-01 | TxID Gen           | counter=1    | WB0000001   | PASS   |
| UT-02 | TxID Gen           | counter=171  | WB0000171   | PASS   |
| UT-03 | Revenue Airtel     | 964*0.05     | 48.2        | PASS   |
| UT-04 | Revenue MTN        | 582*0.06     | 34.92       | PASS   |
| UT-05 | Booth Filter Wina4 | Select Wina4 | 3 services  | PASS   |
| UT-06 | Booth Filter Wina1 | Select Wina1 | 5 services  | PASS   |
| UT-07 | Location           | Wina3        | Kabwata     | PASS   |
| UT-08 | Rate Display       | MTN Money    | 0.06        | PASS   |
| UT-09 | Validation         | Empty form   | Toast error | PASS   |
| UT-10 | Validation         | Amount=0     | Error       | PASS   |

|       |               |                |             |      |
|-------|---------------|----------------|-------------|------|
| UT-11 | API POST      | Wina1,500      | 201 Created | PASS |
| UT-12 | API Invalid   | Invalid booth  | 400 error   | PASS |
| UT-13 | API Dashboard | GET /dashboard | JSON totals | PASS |
| UT-14 | API Booths    | GET /booths    | 6 booths    | PASS |

### Regression Testing - 9 Cases

| ID    | Feature    | Change         | Result        | Status |
|-------|------------|----------------|---------------|--------|
| RT-01 | Chart.js   | CDN fix        | Charts render | PASS   |
| RT-02 | Navigation | Backend added  | Tabs OK       | PASS   |
| RT-03 | Dropdowns  | database.js    | 6 booths OK   | PASS   |
| RT-04 | Responsive | Tailwind       | 768px OK      | PASS   |
| RT-05 | Toasts     | Error handling | Toasts OK     | PASS   |
| RT-06 | Revenue    | Client+Server  | Match OK      | PASS   |
| RT-07 | Table      | Fetch API      | Table OK      | PASS   |
| RT-08 | Export     | Persistence    | CSV OK        | PASS   |
| RT-09 | TxID       | Counter        | Increment OK  | PASS   |

Summary: 23 Tests | 23 Passed | 0 Failed | 100% Pass Rate

## Brief Report (1000 words) - Tools, Implications, Controls - Requirement k) (20 marks)

### 1. Main Tools Used to Develop My Web Application (350 words)

I developed Wina Bwangu using modern web technologies that I have learned. For the client-side application, I used HTML5 for structure, Tailwind CSS for styling, and Vanilla JavaScript for interactivity. I chose Tailwind CSS because it allowed me to create my glassmorphism and gradient design that I liked in my first version maintained. I used Font Awesome for icons and Chart.js 4.4.0 via jsDelivr CDN for my pie chart and bar charts that show performance.

For the service-side application, I used Node.js 20 LTS as my runtime and Express.js 4.18.2 as my framework. I created server/index.js to handle API requests with endpoints: GET /api/booths, GET /api/services, GET /api/transactions, GET /api/dashboard, and POST /api/transactions. I used CORS middleware to allow my frontend to communicate with my backend. I used path module to serve my static files from public folder.

For the database, I created server/database.js as my database layer that simulates MySQL concept as required for Exercise 3. My database contains: 6 booths (Wina1 to Wina6) with locations (Lusaka CPD, Chilenje, Kabwata, Mandevu, Matero, Kanyama), 5 services (Airtel Money 0.05, MTN Money 0.06, Zamtel Money 0.055, ZESCO, DSTV), monthly limits per service, and 170 transactions from Table 3 Appendix 1 as required. My TransactionID generation follows Table 3 format: WB + 7-digit number (WB0000001). I deployed my application to two platforms: GitHub Pages for my live link <https://thewalkingdollar.github.io/wina-bwangu/> and Vercel for my backend APIs.

### 2. Possible Implications of the Outputs of My Web Application (350 words)

My web application outputs have several business implications for Wina Bwangu FINTECH. First, my Dashboard calculations help agents determine whether there is enough available credit for transactions. For example, if Airtel Money has monthly limit K100,000 and cumulative total is K85,000, remaining is K15,000 - if client requests K20,000, agent knows it will exceed limit. This prevents failed transactions and maintains customer trust.

Second, my cumulative revenue per booth performance helps management identify best and worst performing booths. If Wina1 generates K5,200 revenue vs Wina4 K1,800, management can investigate why Wina4 underperforms - maybe location, agent training, or marketing. My frequency analysis shows which services are most requested at each booth, helping with inventory planning.

Third, my Total Revenue and Total Capital calculations show overall business health. If Total Capital required is K245,000 per month but Total Revenue is only K12,000 (4.9% margin), business needs to evaluate if sustainable. My pie chart visualizes this split for quick decision making. My data bars showing tax obligations help with ZRA compliance - if transactions show high tax liability, company can plan tax payments.

Fourth, my transaction history with auto-generated IDs provides audit trail for accountability. Each transaction has WB ID, booth, location, service, amount, revenue - useful for fraud detection. If one booth shows unusual pattern (many high-value transactions at odd hours), system flags for investigation.

### 3. Possible Controls That My Company Might Require to Optimise Operations (300 words)

To optimise operations based on my web application outputs, Wina Bwangu needs several controls. First, financial controls: I recommend daily reconciliation of my dashboard totals vs actual cash. My system shows cumulative totals, but physical cash counts should match. Set alerts when remaining limit falls below 20% to trigger float top-up. Implement dual approval for transactions above K5,000 to reduce fraud.

Second, operational controls: Use my booth performance data to rotate agents between high and low performing booths for cross-training. My frequency data shows peak times - schedule more agents during busy periods. Standardize my dropdown validation - I already prevent empty submissions with toast notifications, but add server-side validation too.

Third, technical controls: My current database is in-memory (server/database.js). For production, migrate to real MySQL with backups, as I demonstrated concept. Add authentication so only authorized agents access my system - currently open. Implement rate limiting on my POST /api/transactions to prevent spam. Add logging for all transactions with timestamps and user IDs.

Fourth, compliance controls: My tax obligation bars should trigger monthly tax reports for ZRA. Keep 7-year transaction records as per Zambian law. My TransactionID auto-generation ensures uniqueness - add checksum digit to prevent manual ID manipulation. Regularly test my system as I did with 23 tests (100% pass) - run unit tests before each deployment.

In conclusion, my web application provides comprehensive tools for managing mobile money booths, but controls are needed to ensure data accuracy, prevent fraud, and maintain compliance. My live link <https://thewalkingdollar.github.io/wina-bwangu/> demonstrates working prototype meeting all requirements a) to k).

**Word Count: ~1000 words**

**Live Link: <https://thewalkingdollar.github.io/wina-bwangu/> | GitHub: <https://github.com/thewalkingdollar/wina-bwangu> |**

**Student: Blessed Chisanga 202202741**

Live Link: <https://thewalkingdollar.github.io/wina-bwangu/> | Vercel: <https://wina-bwangu.vercel.app> | Prepared by Blessed Chisanga (202202741) |  
September 2026